

UNIVERSIDAD ARGENTINA DE LA EMPRESA | CARRERAS DE
INFORMÁTICA

TP 05 · Desarrollo de Aplicaciones II · TPO

Dashboard web de telemetría robótica

*Integración de servicios, visualización de datos en tiempo real y exportación —
versión web del monitor de motores Unitree*

MODALIDAD

TPO — Obligatorio

STACK

Python + HTML/JS

ROBOT

Go2 o G1

TRABAJO

Equipos de 2–3 personas

NIVEL

Intermedio-Alto

1. Título del Trabajo Práctico

Dashboard web de telemetría robótica: versión web del monitor de motores Unitree

2. Materia

Desarrollo de Aplicaciones II — Carreras de Informática, UADE

3. Duración Estimada

Taller de inicio (2 horas): Presentación del problema, análisis de la arquitectura backend/frontend, conexión con la API provista y construcción de los primeros paneles del dashboard.

Extensión como TPO evaluable: Completar los cuatro paneles del dashboard, implementar los gráficos en tiempo real, la exportación CSV y la integración con WebSocket para actualización sin polling. Duración

estimada: 2 a 3 semanas fuera del horario de clase.

4. Fundamentación

En el ámbito de la robótica y los sistemas embebidos, los datos de telemetría son fundamentales para supervisar el estado de los equipos, detectar anomalías y tomar decisiones en tiempo real. Herramientas como el monitor de motores Unitree (*electroSim*) —desarrollado en la cátedra con Python y PyQt6— permiten visualizar en tiempo real el ángulo, velocidad, torque y temperatura de cada motor del robot, junto con datos del sistema inercial (IMU), la batería (BMS) y las fuerzas de contacto por pata.

Este TPO propone construir la versión web de ese mismo monitor: una aplicación cliente-servidor donde un backend en Python expone los datos del robot como un servicio REST y un frontend en HTML/JS consume esos datos para construir un dashboard interactivo en el navegador. La división entre backend (proveedor de datos) y frontend (visualizador) es el patrón central de la materia: el alumno experimenta directamente qué significa integrar componentes independientes a través de una interfaz de servicio definida.

El modo demo del servidor —sin necesidad del robot físico— permite desarrollar y probar el dashboard completo durante el taller, replicando la estrategia usada en la app de escritorio original.

5. Objetivos Generales

- Construir una arquitectura cliente-servidor donde un servicio Python expone telemetría del robot y un frontend web la consume y visualiza.
- Aplicar patrones de integración (Adapter, Observer, Facade) para desacoplar la fuente de datos del robot de la capa de presentación web.
- Implementar visualización de datos en tiempo real con gráficos de línea históricos, indicadores semafóricos y tablas actualizables.
- Incorporar funcionalidades de exportación de datos (CSV) replicando la capacidad de captura y exportación presente en el monitor de escritorio.

6. Objetivos Específicos

- Levantar el servidor FastAPI provisto por la cátedra (basado en la arquitectura de *electroSim*) y verificar que responde correctamente a los endpoints de telemetría.
- Implementar el **panel de motores**: tabla con temperatura (semáforo verde/amarillo/rojo igual al monitor de escritorio), ángulo, velocidad y torque de cada motor.

- Implementar el **panel IMU**: visualización de roll, pitch y yaw en tiempo real con indicadores numéricos y gráfico de línea histórico.
- Implementar el **panel BMS**: nivel de carga (%), corriente, temperatura y voltajes individuales de celda.
- Implementar el **panel de fuerzas por pata**: indicadores de contacto para las cuatro patas (FR, FL, RR, RL).
- Implementar la **exportación CSV** de los datos capturados durante la sesión, replicando la funcionalidad de captura de *electroSim*.
- Reemplazar el polling por **WebSocket** (extensión) para recibir los datos del servidor en modo push, eliminando la latencia del intervalo fijo.

7. Contenidos de la Materia que se Trabajan

- **Implementación de patrones estructurales:** Adapter (normalizar datos de Go2 —12 motores— y G1 —29 motores— en una estructura JSON común), Observer (suscripción del frontend a los datos del servidor), Facade (endpoint `/telemetria` como interfaz unificada de todos los subsistemas del robot).
- **Tipos de integración:** integración por datos (REST polling cada 500ms), integración por eventos (WebSocket para recepción en modo push), integración por archivo (exportación CSV de la sesión).
- **Alternativas de integración:** análisis comparativo entre REST polling y WebSocket en términos de latencia, carga del servidor y complejidad de implementación para el caso de telemetría de robot.
- **Introducción a las arquitecturas orientadas a servicios (SOA):** el backend FastAPI como proveedor de servicios de telemetría; el dashboard como consumidor independiente que puede correr en cualquier dispositivo de la red.
- **Tecnologías y frameworks para construcción de servicios web:** FastAPI para el backend (exposición de endpoints REST y WebSocket); `fetch` API y la API WebSocket nativa del navegador para el frontend.
- **Integración de componentes propios o de terceros:** Chart.js (gráficos históricos de línea, componente de tercero); tabla de motores con semáforo de temperatura (componente propio); lógica de captura y exportación CSV (componente propio).
- **Arquitectura de aplicaciones web:** separación estricta entre backend (Python/FastAPI) y frontend (HTML/JS); comunicación exclusivamente a través de la API definida; independencia de implementación de cada capa.
- **JavaScript moderno:** `async/await`, `fetch`, `setInterval`, manipulación del DOM, WebSocket API nativa.

8. Descripción General de la Actividad

La cátedra provee un servidor FastAPI en Python (basado en la arquitectura del monitor de escritorio *electroSim*) que expone los datos de telemetría del robot como endpoints REST y WebSocket. El servidor tiene un **modo demo** que simula datos realistas de un robot en movimiento —ángulos variables, temperatura en ascenso, IMU oscilante— sin necesitar el robot físico.

Los equipos construyen el frontend web (HTML/CSS/JavaScript) que consume esos datos y los presenta en un dashboard con cuatro paneles: motores, IMU, BMS y fuerzas por pata. El dashboard replica visualmente las capacidades del monitor de escritorio *electroSim*: la misma codificación de color por temperatura, los mismos datos por motor, la misma lógica de captura y exportación CSV.

La arquitectura cliente-servidor que resulta puede desplegarse en la red del laboratorio y ser accedida desde cualquier navegador —PC, tablet o smartphone— sin instalar nada en el cliente, a diferencia de la app de escritorio que requiere Python y PyQt6 instalados.

9. Modalidad de Trabajo

Organización: Equipos de 2 a 3 personas. Roles sugeridos: un integrante lidera el backend (configuración del servidor FastAPI, endpoints, modo demo), otro el frontend (HTML/CSS/JS, paneles, Chart.js) y el tercero la integración y exportación CSV.

Modalidad tecnológica:

- **Modo demo (recomendado para el taller):** El servidor FastAPI corre en modo simulación sin robot físico. Los datos varían de forma realista para poder probar todos los paneles del dashboard.
- **Con robot real:** El servidor se conecta al robot via DDS/SDK. El frontend no cambia; solo cambia la fuente de datos del backend. Disponible en laboratorio bajo supervisión docente.

10. Recursos Necesarios

Backend (provisto por la cátedra)

- Python 3.10+ con FastAPI y Uvicorn: `pip install fastapi uvicorn`
- Servidor de telemetría (`robot_telemetry_server.py`) con modo demo integrado — provisto por la cátedra, basado en la arquitectura de *electroSim*.
- Opcional (robot real): Unitree SDK 2 Python + CycloneDDS.

Frontend (desarrollado por los alumnos)

- HTML5 + CSS3 + JavaScript (sin framework obligatorio; se admite React o Vue si el equipo lo prefiere).
- Chart.js (CDN, sin instalación): gráficos de línea históricos para IMU, temperatura y voltajes de celda.
- VS Code como editor.

Archivos provistos por la cátedra

- `robot_telemetry_server.py` : servidor FastAPI con endpoints REST y WebSocket, modo demo con simulación de Go2 y G1.
- `tp05_dashboard_base.html` : estructura HTML del dashboard con estilos base y los cuatro paneles vacíos para completar.
- `tp05_api_docs.pdf` : documentación de todos los endpoints con formato de respuesta JSON.

11. Desarrollo del Taller de 2 Horas

Tiempo	Actividad del Docente	Actividad de los Alumnos
00–15 min	Presenta <i>electroSim</i> (monitor de escritorio) en vivo: muestra datos de motores, IMU y BMS en tiempo real. Plantea la pregunta: "¿Cómo haríamos esto para que funcione en el navegador?"	Observan el monitor de escritorio. Identifican qué datos hay que mostrar y qué diferencia existe entre una app de escritorio y una web.
15–30 min	Explica la arquitectura: servidor FastAPI como puente entre el robot (o el simulador) y el navegador. Muestra los endpoints disponibles usando la UI de Swagger (/docs). Levanta el servidor en modo demo.	Levantán el servidor en su PC, abren <code>http://localhost:8000/docs</code> y prueban manualmente los endpoints desde el navegador. Verifican el formato JSON de respuesta.
30–50 min	Guía la implementación del panel de motores: tabla HTML con las columnas motor, temperatura (celda con color), ángulo, velocidad y torque. Explica la lógica del semáforo de temperatura (igual al monitor de escritorio).	Implementan la función <code>actualizarMotores(data)</code> que llena la tabla con los datos del endpoint <code>/motores</code> . Implementan la función de color por temperatura.
50–80 min	Propone agregar Chart.js para el gráfico de temperatura histórica. Explica el búfer circular de 300 muestras (igual al historial de 30 segundos de <i>electroSim</i>). Muestra cómo iniciar el polling con <code>setInterval</code> .	Integran Chart.js, crean el gráfico de líneas para temperatura de motores seleccionados, implementan el búfer circular y lo conectan al polling.
80–105 min	Propone el panel IMU y el panel BMS. Recorre los puestos, resuelve dudas. Para quienes terminaron: propone implementar el botón de exportación CSV.	Implementan los paneles de IMU (roll, pitch, yaw) y BMS (batería, corriente, celdas). Los más avanzados agregan la captura de muestras y el botón de descarga CSV.
105–120 min	Muestra el dashboard de un equipo proyectado. Reflexión: ¿qué ventaja tiene la versión web respecto al monitor de escritorio? ¿Qué se pierde?	Presentan su avance. Comparan su dashboard con el monitor de escritorio. Discuten trade-offs: instalación vs. acceso universal, latencia de polling vs. DDS.

12. Consigna para los Alumnos

Trabajo Práctico Obligatorio N° 5 — Desarrollo de Aplicaciones II

Construyan un dashboard web que replique en el navegador las capacidades del monitor de motores Unitree (*electroSim*). El backend (servidor FastAPI) es provisto por la cátedra; ustedes construyen el frontend. El dashboard debe tener cuatro paneles funcionales:

1. **Panel de motores:** tabla con temperatura (semáforo: verde $<40^{\circ}\text{C}$, amarillo $40\text{--}60^{\circ}\text{C}$, rojo $>60^{\circ}\text{C}$, igual a *electroSim*), ángulo ($^{\circ}$), velocidad angular (rad/s) y torque estimado (N·m) para cada motor del robot seleccionado (12 motores para Go2, 29 para G1).
2. **Panel IMU:** valores numéricos de roll, pitch y yaw con un gráfico de línea histórico de al menos 60 muestras.
3. **Panel BMS:** nivel de carga (%), corriente (mA), temperatura de batería y voltajes individuales de celda.
4. **Panel de fuerzas por pata:** indicadores de contacto para FR (delantera derecha), FL (delantera izquierda), RR (trasera derecha) y RL (trasera izquierda).

Requisitos transversales: actualización automática mediante polling a `/telemetria` (máx. 500ms de intervalo) o WebSocket; botón de **Capturar muestra** que guarda el estado actual; botón de **Exportar CSV** que descarga todas las muestras capturadas. El dashboard debe funcionar correctamente en modo demo del servidor.

13. Desarrollo Técnico Esperado

El servidor FastAPI provisto expone el endpoint `GET /telemetria` que retorna un objeto JSON con los cuatro subsistemas del robot: `motores` (array), `imu`, `bms` y `fuerzas`. En modo demo, los valores varían de forma realista simulando un robot en movimiento: los ángulos oscilan con amplitudes distintas por articulación (cadera, muslo, pantorrilla), la temperatura sube gradualmente durante el primer minuto y la IMU oscila con frecuencia de paso.

El panel de motores implementa la misma lógica de semáforo de temperatura que *electroSim*: la celda de temperatura de cada fila cambia de color según el umbral (`verde` $< 40^{\circ}\text{C}$, `amarillo` $40\text{--}60^{\circ}\text{C}$, `rojo` $> 60^{\circ}\text{C}$). Esta información visual permite identificar de un vistazo qué motores están en riesgo de sobrecalentamiento.

El gráfico histórico de Chart.js usa un búfer circular de 300 entradas (equivalente a 30 segundos a 10 Hz, igual al historial de *electroSim*): al llenarse, elimina la muestra más antigua y agrega la nueva, manteniendo

siempre una ventana deslizable sin recrear el gráfico.

La exportación CSV replica la funcionalidad de captura de *electroSim*: cada vez que el usuario hace clic en "Capturar muestra", se agrega un registro al array de sesión con el timestamp y los valores actuales de todos los paneles. Al exportar, se genera un CSV en el navegador usando la API `Blob` y se descarga sin pasar por el servidor.

14. Ejemplo de Estructura de Código

Backend — `robot_telemetry_server.py` (provisto por la cátedra)

```

# Servidor FastAPI de telemetría - modo demo basado en electroSim
# Provisto por la cátedra. Los alumnos no modifican este archivo.

import math, time, random
from fastapi import FastAPI, WebSocket
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(title="Unitree Telemetry API")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"])

inicio = time.time()

def simular_motor(i: int) -> dict:
    t = time.time() - inicio
    angulo = math.sin(t * 1.2 + i * 0.5) * 30
    velocidad = math.cos(t * 1.2 + i * 0.5) * 5
    torque = abs(math.sin(t * 0.8 + i)) * 20
    temp_base = 28 + i * 0.5
    temperatura = temp_base + min(t / 60, 1) * 25 + random.uniform(-0.5, 0.5)
    return {"id": i, "angulo": round(angulo, 2),
            "velocidad": round(velocidad, 3),
            "torque": round(torque, 2),
            "temperatura": round(temperatura, 1)}

def simular_imu() -> dict:
    t = time.time() - inicio
    return {"roll": round(math.sin(t * 0.3) * 5, 2),
            "pitch": round(math.sin(t * 0.5) * 8, 2),
            "yaw": round(math.sin(t * 0.1) * 15, 2),
            "ax": round(random.gauss(0, 0.05), 3),
            "ay": round(random.gauss(0, 0.05), 3),
            "az": round(9.81 + random.gauss(0, 0.02), 3)}

def simular_bms() -> dict:
    t = time.time() - inicio
    soc = max(20, 95 - t / 600)
    return {"soc": round(soc, 1), "corriente": round(1200 + random.uniform(-50, 50), 0),
            "temperatura": round(32 + random.uniform(-1, 1), 1),
            "celdas": [round(3.7 + random.uniform(-0.05, 0.05), 3) for _ in range(10)]}

@app.get("/telemetria")
def telemetria(modelo: str = "go2"):
    n_motores = 12 if modelo == "go2" else 29
    return {
        "modelo": modelo,
        "ts": round(time.time() - inicio, 2),
        "motores": [simular_motor(i) for i in range(n_motores)],
    }

```

```
"imu":      simular_imu(),
"bms":      simular_bms(),
"fuerzas":  {"FR": random.choice([0,1]), "FL": random.choice([0,1]),
             "RR": random.choice([0,1]), "RL": random.choice([0,1])},
}

@app.websocket("/ws")
async def websocket_endpoint(ws: WebSocket):
    await ws.accept()
    while True:
        import asyncio
        await ws.send_json(telemetry())
        await asyncio.sleep(0.1)  # 10 Hz, igual que electroSim
```

Frontend — función de actualización de motores con semáforo de temperatura

```

// dashboard.js - Panel de motores con semáforo de temperatura (igual a electroSim)

const HIST_SIZE = 300; // 30 s a 10 Hz - igual al historial de electroSim
const POLL_MS = 500;
const API = 'http://localhost:8000';

let modelo = 'go2';
let histTemp = {}; // { motorId: [valores] }
let muestras = []; // capturas manuales para exportar CSV
let chart = null;

// --- Semáforo de temperatura (igual a electroSim) ---
function colorTemp(t) {
  if (t < 40) return '#4CAF50'; // verde
  if (t < 60) return '#FFC107'; // amarillo
  return '#F44336'; // rojo
}

// --- Actualizar tabla de motores ---
function actualizarMotores(motores) {
  const tbody = document.querySelector('#tablaMotores tbody');
  tbody.innerHTML = '';
  motores.forEach(m => {
    const tr = document.createElement('tr');
    tr.innerHTML = `
      ${m.id}

      ${m.temperatura} °C

      ${m.angulo} °
      ${m.velocidad} rad/s
      ${m.torque} N·m`;
    tbody.appendChild(tr);

    // Actualizar búfer circular histórico
    if (!histTemp[m.id]) histTemp[m.id] = [];
    histTemp[m.id].push(m.temperatura);
    if (histTemp[m.id].length > HIST_SIZE) histTemp[m.id].shift();
  });
}

// --- Actualizar panel IMU ---
function actualizarIMU(imu) {
  document.getElementById('roll').textContent = imu.roll + ' °';
  document.getElementById('pitch').textContent = imu.pitch + ' °';
  document.getElementById('yaw').textContent = imu.yaw + ' °';
}

```

```

// --- Actualizar panel BMS ---
function actualizarBMS(bms) {
  document.getElementById('soc').textContent = bms.soc + ' %';
  document.getElementById('bmsI').textContent = bms.corriente + ' mA';
  document.getElementById('bmsT').textContent = bms.temperatura + ' °C';
  const celdasDiv = document.getElementById('celdas');
  celdasDiv.innerHTML = bms.celdas
    .map((v, i) => `
      C${i+1}: ${v} V `).join(' ');
}

// --- Actualizar fuerzas por pata ---
function actualizarFuerzas(f) {
  ['FR', 'FL', 'RR', 'RL'].forEach(pata => {
    const el = document.getElementById('pata' + pata);
    el.style.background = f[pata] ? '#4CAF50' : '#eee';
    el.textContent = pata + (f[pata] ? ' ✓' : ' -');
  });
}

// --- Capturar muestra para exportar ---
function capturarMuestra(data) {
  muestras.push({ ts: data.ts, ...data.imu,
    soc: data.bms.soc, corriente: data.bms.corriente });
  document.getElementById('nMuestras').textContent = muestras.length;
}

// --- Exportar CSV (sin pasar por el servidor, igual a electroSim) ---
function exportarCSV() {
  if (!muestras.length) return;
  const cols = Object.keys(muestras[0]);
  const csv = [cols.join(','),
    ...muestras.map(r => cols.map(c => r[c]).join(',')).join('\n')];
  const a = Object.assign(document.createElement('a'), {
    href: URL.createObjectURL(new Blob([csv], { type: 'text/csv' })),
    download: 'telemetria_' + Date.now() + '.csv'
  });
  a.click();
}

// --- Polling principal ---
async function poll() {
  try {
    const data = await fetch(`${API}/telemetria?modelo=${modelo}`).then(r => r.json());
    actualizarMotores(data.motores);
    actualizarIMU(data.imu);
    actualizarBMS(data.bms);
  }
}

```

```
    actualizarFuerzas(data.fuerzas);  
  } catch { /* sin conexión - mantener últimos valores */ }  
}  
  
setInterval(poll, POLL_MS);  
poll();
```

15. Entregables

- **Repositorio o carpeta comprimida** con el código fuente del frontend (`dashboard.html` , `dashboard.js` , `dashboard.css`). El servidor no se entrega (es provisto por la cátedra).
- **Capturas de pantalla** del dashboard funcionando en modo demo con los cuatro paneles visibles y la tabla de motores con semáforo de temperatura activo.
- **Archivo CSV exportado** durante una sesión de prueba de al menos 10 muestras capturadas manualmente.
- **Documento técnico (máx. 2 páginas)**: descripción de la arquitectura (endpoints consumidos, estructura del frontend), decisiones de diseño tomadas para los paneles y análisis comparativo entre polling y WebSocket para este caso de uso.

16. Criterios de Evaluación

- Los cuatro paneles están implementados y muestran datos actualizados correctamente.
- El semáforo de temperatura del panel de motores replica fielmente la lógica de *electroSim* (verde/amarillo/rojo con los mismos umbrales).
- Los gráficos de línea histórica usan un búfer circular y se actualizan sin recrear el componente.
- La captura de muestras funciona y el CSV exportado contiene todos los campos solicitados.
- El dashboard maneja la pérdida de conexión con el servidor sin crashear.
- El documento técnico demuestra comprensión de la arquitectura cliente-servidor y de los patrones de integración aplicados.

17. Rúbrica de Evaluación

Criterio	Indicadores de logro	Puntaje máximo
Panel de motores	Tabla completa con semáforo de temperatura (colores correctos), ángulo, velocidad y torque. Se actualiza automáticamente. Funciona para Go2 (12 motores) y G1 (29 motores).	25
Paneles IMU y BMS	IMU muestra roll, pitch y yaw actualizados con gráfico histórico de al menos 60 muestras. BMS muestra SOC, corriente, temperatura y voltajes de celda individuales.	25
Panel de fuerzas y estado general	Indicadores de contacto por pata (FR/FL/RR/RL) activos. El dashboard indica visualmente cuando hay pérdida de conexión con el servidor.	15
Captura y exportación CSV	El botón de captura registra la muestra correctamente. El CSV exportado contiene timestamp, IMU y BMS. La exportación ocurre en el navegador sin pasar por el servidor.	20
Calidad técnica y documento	Código modular con funciones separadas por panel. Sin lógica duplicada. El documento describe la arquitectura, patrones usados y el análisis polling vs. WebSocket.	15
TOTAL		100

18. Medidas de Seguridad

Modalidad demo / simulada: No hay riesgos físicos. El servidor corre localmente y no controla hardware real. El CORS del servidor está configurado en modo abierto (`allow_origins=["*"]`) solo para uso en red de laboratorio cerrada; no debe exponerse a internet.

Si el servidor se conecta al robot físico:

- El servidor de telemetría es de solo lectura: expone datos del robot pero no envía comandos de movimiento. No hay riesgo de movimiento accidental desde el dashboard.

- La conexión al robot requiere acceso a la red local del laboratorio bajo supervisión docente.
- El dashboard no debe modificarse para incluir endpoints de control sin autorización de la cátedra.

19. Posibles Extensiones del TP

- **Extensión 1 — WebSocket:** Reemplazar el polling por WebSocket (`new WebSocket('ws://localhost:8000/ws')`) para recibir datos en modo push a 10 Hz, igual que la frecuencia interna de *electroSim*. Analizar la diferencia de latencia y carga con respecto al polling.
- **Extensión 2 — Selector de motores en el gráfico:** Un checkbox por motor permite elegir cuáles curvas de temperatura se muestran en el gráfico histórico, replicando la leyenda interactiva de *electroSim*.
- **Extensión 3 — Alertas automáticas:** Cuando la temperatura de cualquier motor supera 60°C, el dashboard muestra una notificación visual (toast) y registra el evento en el CSV automáticamente sin que el usuario tenga que capturar manualmente.
- **Extensión 4 — Modo multirobot:** El dashboard puede conectarse a múltiples instancias del servidor (una por robot) y cambiar entre ellas con un selector, mostrando el modelo activo.
- **Extensión 5 — Visualización 3D del IMU:** Un canvas WebGL (usando Three.js) muestra un modelo simplificado del robot que se orienta en tiempo real según los valores de roll, pitch y yaw del IMU.

20. Resultado Esperado

Al finalizar el TPO, cada equipo habrá construido un dashboard web funcional que replica en el navegador las capacidades de monitoreo del motor de escritorio *electroSim*: visualización en tiempo real de motores, IMU, BMS y fuerzas por pata, con semáforo de temperatura, gráficos históricos y exportación CSV. La arquitectura cliente-servidor resultante puede desplegarse en la red del laboratorio y accederse desde cualquier dispositivo sin instalación, demostrando concretamente el valor de las arquitecturas orientadas a servicios frente a las aplicaciones de escritorio monolíticas.

21. Cierre y Reflexión para los Alumnos

La aplicación de escritorio que sirvió de inspiración para este TP —*electroSim*— hace exactamente lo mismo que el dashboard que construyeron, pero de una manera radicalmente diferente: necesita Python, PyQt6 y el SDK del robot instalados en cada máquina que lo use. El dashboard web que construyeron corre en cualquier navegador de la red sin instalar nada.

Eso no significa que uno sea mejor que el otro: la app de escritorio tiene acceso directo al SDK del robot vía DDS, puede leer datos a 100 Hz sin la sobrecarga de HTTP, y funciona sin red Wi-Fi. El dashboard web puede abrirse en un tablet, mostrarse en una pantalla grande del laboratorio y ser accedido simultáneamente por varios usuarios. Son soluciones para contextos distintos.

Saber cuándo usar una arquitectura de servicio y cuándo una app monolítica, y entender exactamente qué se gana y qué se pierde en cada caso, es lo que diferencia a un desarrollador que simplemente usa frameworks de uno que diseña sistemas.